

A Novel Structure of the Smith-Waterman Algorithm for Efficient Sequence Alignment

Saad Khan Zahid, Laiq Hasan

Department of Computer Systems Engineering
University of Engineering & Technology Peshawar,
Pakistan

saadzahid@live.com
laiqhasan@uetpeshawar.edu.pk

Asif Ali Khan, Salim Ullah

Department of Computer Systems Engineering
University of Engineering & Technology Peshawar,
Pakistan

[\[asif.ali, saleemullah\]@uetpeshawar.edu.pk](mailto:[asif.ali, saleemullah]@uetpeshawar.edu.pk)

Abstract—The emergence of bioinformatics has led to many new discoveries in living organisms. These discoveries would not have been possible without the developments made in the sequence alignment techniques. Many sequence alignment algorithms were developed to make the alignment process fast and accurate. However, the more precise algorithms take longer than their less precise counterparts. Researchers came with innovative approaches to combat the time consuming constraint. Their aim was to speed up the computational process by using more efficient implementations of the algorithms using state-of-the-art hardware platforms. Smith Waterman (SW) algorithm, being the most accurate in the alignment process, has been implemented on various high performance computing platforms for the same purpose. However, the intrinsic structure of the algorithm has got little attention. In this paper, we present a novel structure of the SW algorithm that takes less number of cycles at the cost of utilizing a minimal amount of extra hardware resources as compared to its existing form. The newly proposed architecture achieves up to 25% performance gain.

Keywords—Bioinformatics, FPGA, GPUs, Hardware Accelerators, Smith Waterman, Sequence Alignment, Systolic Arrays

I. INTRODUCTION

Life on Earth originated and then evolved from a universal common ancestor approximately 3.8 billion years ago. Repeated speciation and the divergence of life has occurred throughout this time due to shared sets of biochemical and morphological traits, or by shared DNA sequences. In bioinformatics, sequence alignment deals with the comparison of two or more Deoxyribonucleic Acid (DNA), Ribonucleic Acid (RNA) and Protein sequences with each other. The comparison aims to identify regions of similarity that may be a consequence of functional, structural, or evolutionary relationships between the sequences. Such sequences are said to be homologous [1].

Sequence alignment plays an important role in drug design and helps in detecting any abnormal changes in protein sequences to which the drug is subjected. As a result proper drug can be formulated with reduced side effects [2,3,4]. Hence, sequence alignment is playing a vital influential role in development of human life and society.

In most of the research till date, the focus is on optimal

implementation and parallelization of various relevant sequence alignment algorithms e.g. the Smith-Waterman. Almost no work has been done in analyzing these algorithms. The aim of this research is to devise a strategy that reduces the computational time consumed by the SW algorithm. Although many hardware and software accelerations have already been brought into reality, yet there is no such work done to improve the basic structure of algorithms like the Smith-Waterman. We propose a theoretical model that suggests changes in the structure of the algorithm which in turn reduces the execution time at the cost of utilizing extra hardware resources.

The rest of the paper is organized as follows. Section II briefly discusses the related work. Section III describes the SW algorithm and its cell level hardware architecture. Section IV analyses the SW algorithm and presents the modified three cycles architecture. Section V presents the results and compares the existing algorithm with its modified counterpart. Section VI concludes the paper.

II. RELATED WORK

Efficient sequence alignment is one of the most important and challenging activities in bioinformatics. Many algorithms have been proposed previously to perform sequence alignment activities. Some of them are heuristic based e.g. FASTA, BLAST etc. [5,6] which are quite fast but less sensitive and accurate while others e.g. Smith-Waterman [7] and Needleman-Wunsch [8] etc. are dynamic programming based exact algorithms which are more accurate but computationally expensive. Amongst all, Smith Waterman (SW) is the most optimal in finding the local alignments. However, the fact that it is computationally expensive; makes it necessary to implement hardware acceleration methods to make the algorithm viable for practical applications.

With the ever increasing volume of data in the bioinformatics databases, the time required for sequence alignment process is also on the exponential rise. Researchers from various backgrounds are working hard to accelerate the alignment process using state of the art hardware and software tools. In [9], the SW algorithm has been implemented on multiple cores using *Open Multi Processing (OpenMp)* [10] programming paradigm. M. Noorian et al. in [11], present the parallel implementation of the same in OpenMp, *Message*

Passing Interface (MPI) [12] and the hybrid of OpenMp and MPI. Experimental results showed that the hybrid model which exploited both coarse-grain and fine-grain parallelism achieved better performance than pure OpenMp and MPI implementations.

Many-core machines and hardware accelerators have remained an attractive choice for reducing processing time of the SW algorithm. Large amount of computations and memory is required for processing the SW algorithm; therefore, memory access time also becomes a bottleneck while implementing the same on *Graphical Processing Unit (GPU)* [13]. To overcome this problem, a two dimensional cache based approach has been used that divides the algorithm in two stages and achieves better performance than CUDASW++ 2.0. Another massively parallel implementation of the SW algorithm on Nvidia GPU using CUDA programming environment is presented in [14].

Field Programmable Gate Arrays (FPGA) having excessive hardware resources have also been used in many different ways to accelerate SW algorithm. In [15], a systolic array has been used for forward process i.e. matrix fill-up process and a separate unit has been used for *Trace Back (TB)* step. A cyclic systolic array based approach has been presented in [16] which provides the same speed and take considerably less hardware resources. The authors of [17] have presented different optimization techniques for implementation of SW algorithm on FPGA. One of the most time consuming operations in FPGA implementation is the matrix calculation operation for determining the maximum score for finding the best sequence alignment. These matrices are implemented using *Smith-Waterman Processing Elements (SWPE)*. This paper discusses four different methods for mapping DNA sequences to SWPEs. The results of the experiments show that the conventional placement of SWPEs gives the best performance results. The lattice based deployment of SWPEs gives worst performance results.

NCBI BLAST alignment tool is used for the design of a configurable and robust FPGA based hardware accelerator for SW [18]. The designed accelerator considerably improves the performance of previously designed BLAST tool. Motivated from the BLAST's word concept, the authors of [19] have presented a hardware and software accelerator for protein sequence alignment. The authors have used pre-selection filter and dynamic gap selector for reducing the SW iterations and hardware complexity respectively. With trivial reduction in alignment accuracy, implementation area and frequency improvements have been presented.

Taner Aruk et al. have proposed BinaryPartialAlign, a fast alignment technique based on SW for genome sequences with large deletions [20]. In their work, they have addressed the inadequacy of most of the available next generation sequence alignment tools by dividing the query sequence into two equal portions and aligning them separately. If only one of the obtained similarity index is greater than the threshold index, the alignment process is repeated again by increasing the length of the portion with bigger similarity index. This process

is repeated until both of the obtained similarity indices are above the threshold index. They have also presented a comparative analysis of BinaryPartialAlign with the incremental partial alignment and the standard SW algorithm in [21]. The comparison of the algorithms was carried out using 900 samples created from mitochondrial (mtDNA) reference. The results show the robustness and lesser execution time of BinaryPartialAlign technique for large query sequences.

III. SMITH WATERMAN ALGORITHM

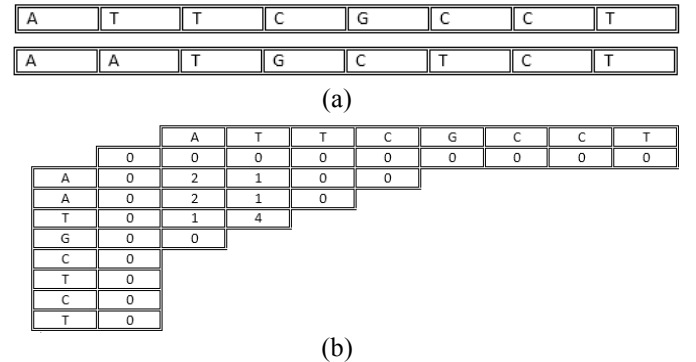
Smith Waterman is the most widely used algorithm for pair wise biological local sequence alignment. As discussed earlier, SW being an exact algorithm gives accurate results at the cost of increased computational complexity. For two sequences of size 'm' and 'n', the computational complexity of the algorithm is $O(mn)$.

As depicted in Figure 1, SW is a 3-steps process that includes initialization of the similarity matrix, matrix fill-up and trace-back. The first row and column of the $(m+1) \times (n+1)$ similarity matrix are initialized to 0 in the initialization step. In the matrix fill-up stage, every cell in the similarity matrix is computed and updated based on Equation 1 where 'S' is similarity score, 'D' is dissimilarity score and 'd' is the gap penalty.

$$H(i, j) = \max \begin{cases} 0 \\ H(i-1, j-1) + S/D \\ H(i-1, j) - d \\ H(i, j-1) - d \end{cases} \quad (1)$$

In the trace-back stage, we obtain the local alignment result. Starting with the cell having maximum value, a trace back path is constructed as shown in Figure 1(c).

Figure 2 shows the number of cycles and hardware components required to compute a single cell value. In the first cycle, the two characters m(j) and n(i) are compared. At the same time, the two values $H(i-1, j)$ and $H(i, j-1)$ are computed as they are independent and can be carried out in parallel. The two values computed in 1st cycle are compared in the 2nd cycle. The addition of the $H(i-1, j-1) + S/D$ also takes place in the 2nd cycle based on the comparator result from cycle 1. The newly computed value is compared with 0 in the 3rd cycle and the final value $H(i, j)$ comes after the last comparison in 4th cycle.



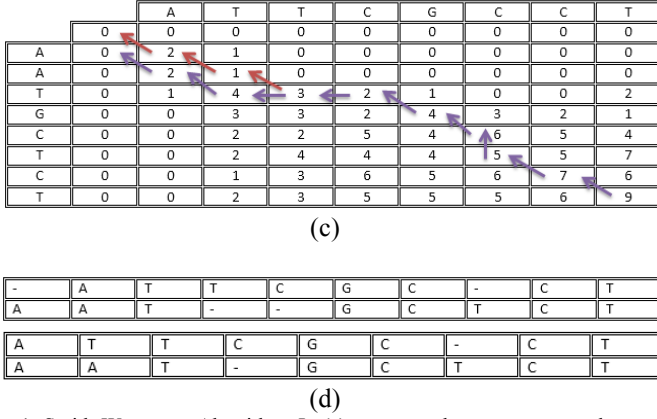


Fig. 1. Smith-Waterman Algorithm. In (a), two sample sequences are shown. (b) shows the anti-diagonal elements that can be computed in parallel (c) Similarity matrix with complete trace-back path (d) Alignment results

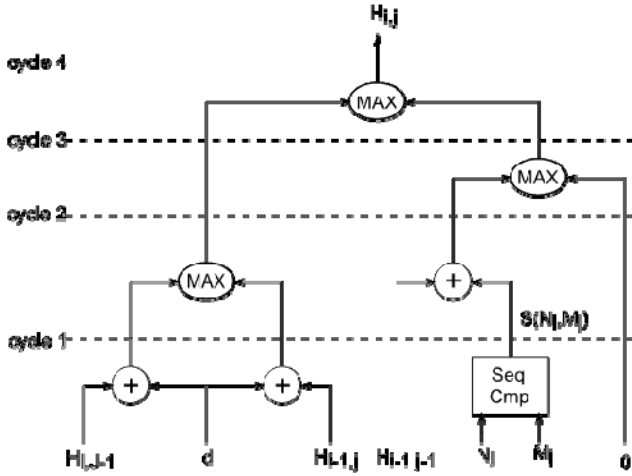


Fig. 2. Cell Architecture of the Smith-Waterman Algorithm

IV. ANALYSIS OF THE SMITH WATERMAN ALGORITHM

The existing implementations of the Smith Waterman Algorithm take four cycles to compute a single value in similarity matrix, as already shown in Figure 2. This section concentrates on increasing the efficiency of SW algorithm by reducing the execution time consumed by each *Processing Element (PE)*. The execution time can be reduced by reducing the number of cycles taken by each PE as

$$T(\text{time taken by each PE}) \propto Nc(\text{number of cycles})$$

SW in Figure 2 uses three Maximum Finding Elements (MAX), three Adders and a Sequence Comparator for finding $H_{i,j}$. We analyzed the effect of adding various number of conditional loops, registers, adders and the MAX, to reduce the number of cycles. The other factors of SW, which can be analyzed for reducing the number of clock cycles, are the input to the algorithm, such as the element of two different sequences N and M , the gap penalty d , the constant 0, the immediate diagonal, upper and left side values $H(i-1)(j-1)$, $H(i-1)(j)$, $H(i)(j-1)$.

Taking all these parameters in consideration, we came up with a new architecture that uses 3 cycles to perform the same task without affecting the overall accuracy. The proposed approach achieves the goal of clock cycle reduction by removing the constant 0. This resulted in reducing the number of adders and MAX components. The new theoretical model is shown in Figure 3. The new model has two adders and two MAX components. By testing different sequences on the processing element of Fig. 3, with $-d \geq 0$ and $D \geq 0$, it has been verified that the number of cycles for computing an element of similarity matrix is reduced to 3.

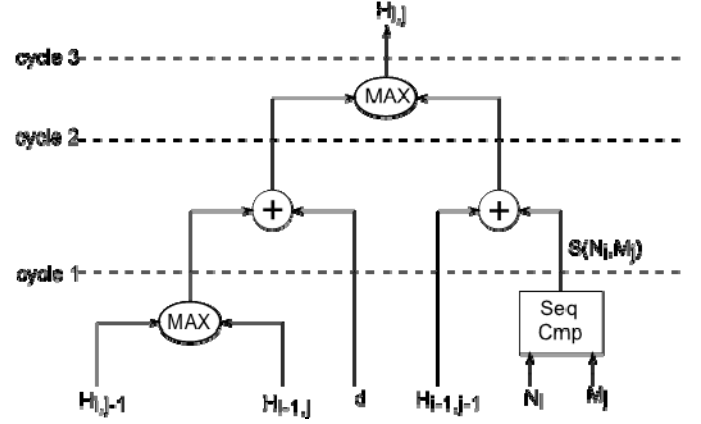


Fig. 3. Cycles design for Smith Waterman Algorithm's Cell

For handling scoring schemes with $-d < 0$ and $D < 0$, the fundamental 4 clock cycles' cell architecture can be used. The one time selection of cell architecture can be decided by comparing scoring schemes at the start of matrix fill-up stage. To compare the scoring schemes, a compare module and two MUXes have been introduced in the datapath as shown in Figure 4. This one time comparison overhead before the start of matrix fill-up stage is mandatory for ensuring the accuracy of the proposed cell architecture. Similarity and dissimilarity scores are checked for negative values by Scoring Scheme Comparator. The Scoring Scheme Comparator using two MUXes enables the suitable cell architecture, by ANDing its clock signal with 1, for similarity matrix values calculation. The disabling of other cell architecture also helps in reducing the overall power consumption of the system. By carrying different test cases on the proposed system, it has been found that the proposed architecture will result in reduced execution time for matrix fill-up stage of the SW algorithm.

V. RESULTS AND DISCUSSION

The modified Smith Waterman algorithm's architecture, presented in Section IV consumes less time as compared to its existing counterpart. However, the proposed system requires an overhead cycle only at the start of the processing for maintaining the accuracy of the algorithm. Table 1 compares the results of the proposed system with the existing one.

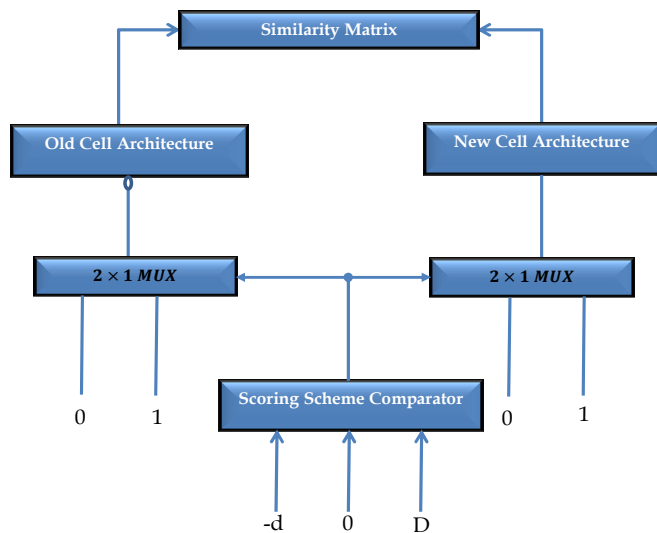


Fig. 4. Cell Architecture Selection for Various Scoring Schemes

TABLE 1 COMPARISON OF 3 CYCLES AND 4 CYCLES ARCHITECTURES

Architecture	Scoring Scheme			
	Scoring Scheme ($D < 0$ & $d < 0$)		Any Other Scoring Scheme	
	Execution Time	Accuracy	Execution Time	Accuracy
4 cycles	Existing + 1 overhead cycle time	100%	Existing + 1 overhead cycle time	100%
3 cycles (modified)	Existing + 1 overhead cycle time	100%	25% less than the existing	100%

VI. CONCLUSION

In this paper, a new architecture for SW algorithm has been presented for the implementation of the matrix fill-up stage in the SW algorithm. The matrix fill-up stage is one of the most time consuming operations in the SW algorithm. The conventional implementation of this phase takes four cycles for computation of each element in the matrix. The newly proposed design reduces this latency to three cycles. However, the proposed design takes one overhead cycle at the start of the processing for ensuring the 100% accuracy. With the newly proposed architecture, the SW algorithm achieves up to 25% speedup.

REFERENCES

- [1] N. Islam, "Faster and efficient algorithm for sequence alignment", Thesis, Department of Computer Science & Engineering BRAC University, 2009.
- [2] J. Cohen, "Bioinformatics: An Introduction for Computer Scientists", ACM Computing Surveys, vol. 36(2), pages 122–158, June 2004.
- [3] C. M. Keet, "Conceptual Modeling for Applied Bioscience", School of Computing, Napier University, Edinburgh, Scotland.

- [4] F. Smith and M. S. Waterman, "Identification of Common Molecular Subsequences", Journal of Molecular Biology, vol. 147, pages 195–197, 1981.
- [5] A. V. Aho; D. S. Hirschberg; J. D. Ullman, "Bounds on the complexity of the longest common subsequence problem" Journal of ACM, 23(1): 1-12, 1976
- [6] S. F. Altschul; W. Gish; W. Miller; E. W. Myers; D.J. Lipman, , "Basic local alignment search tool," Journal of Molecular Biology, 1990, vol. 215: pp. 403-410.
- [7] T. F Smith; M. S. Waterman, "Identification of common molecular subsequences", Journal of Molecular Biology, 1981, vol. 147: pp. 195–197.
- [8] D. Li; M. Becchi, "Abstract: Multiple Pairwise Sequence Alignments with the Needleman-Wunsch Algorithm on GPU," High Performance Computing, Networking, Storage and Analysis (SCC), 2012 SC Companion: , vol., no., pp.1471,1472, 10-16, Nov. 2012
- [9] S. R. Sathe and D.D. Shrimankar, "Parallelization of DNA Sequence Alignment using OpenMP", ICCS 2011, pp. 200-203.
- [10] OpenMP Standards: <http://www.openmp.org/>. (Last Accessed: December 12, 2014).
- [11] M. Noorian; H. Pooshfam; Z. Noorian; R. Abdullah, "Performance enhancement of smith-waterman algorithm using hybrid model: Comparing the MPI and hybrid programming paradigm on SMP clusters," Systems, Man and Cybernetics, 2009. SMC 2009. IEEE International Conference on , vol., no., pp.492,497, 11-14 Oct. 2009
- [12] Open-MPI: <http://www.open-mpi.org/> (Last accessed: December 18, 2014)
- [13] X. Feng; H. Jin; R. Zheng; Z. Shao; L. Zhu, "Implementing Smith-Waterman Algorithm with Two-Dimensional Cache on GPUs," Cloud and Green Computing (CGC), 2012 Second International Conference on , vol., no., pp.25,30, 1-3 Nov. 2012
- [14] L. Ligowski; W. Rudnicki, "An efficient implementation of Smith Waterman algorithm on GPU using CUDA, for massively parallel scanning of sequence databases," Parallel & Distributed Processing, 2009. IPDPS 2009. IEEE International Symposium on , vol., no., pp.1,8, 23-29 May 2009
- [15] Marmolejo-Tejada, J.M.; Trujillo-Olaya, V.; Renteria-Mejia, C.P.; Velasco-Medina, J., "Hardware implementation of the Smith-Waterman algorithm using a systolic architecture," Circuits and Systems (LASCAS), 2014 IEEE 5th Latin American Symposium on , 25-28 Feb. 2014
- [16] P. Chen; C. Wang; X. Li; X. Zhou, "Hardware acceleration for the banded Smith-Waterman algorithm with the cycled systolic array," Field-Programmable Technology (FPT), 2013 International Conference on , vol., no., pp.480,481, 9-11 Dec. 2013
- [17] X. Chang; F.A. Escobar; C. Valderrama; V. Robert, "Optimization Strategies for Smith-Waterman Algorithm on FPGA Platform," Computational Science and Computational Intelligence (CSCI), 2014 International Conference on , vol.1, no., pp.9,14, 10-13 March 2014
- [18] Lam, B.C.; Pascoe, C.; Schaecher, S.; Lam, H.; George, A.D., "BSW: FPGA-accelerated BLAST-Wrapped Smith-Waterman aligner," Reconfigurable Computing and FPGAs (ReConFig), 2013 International Conference on , vol., no., pp.1,7, 9-11 Dec. 2013
- [19] Urgese, G.; Graziano, M.; Vacca, M.; Awais, M.; Frache, S.; Zamboni, M., "Protein alignment HW/SW optimizations," Electronics, Circuits and Systems (ICECS), 2012 19th IEEE International Conference on , vol., no., pp.145,148, 9-12 Dec. 2012
- [20] T. Aruk; D. Ustek; O. Kursun, "A comparative analysis of smith-waterman based partial alignment," Computers and Communications (ISCC), 2012 IEEE Symposium on , vol., no., pp.000250,000252, 1-4 July 2012
- [21] T. Aruk; D. Ustek; O. Kursun, "A novel partial sequence alignment tool for finding large deletions", The Scientific World Journal, doi 10.1100/2012/694813, 2012.